Async Await

Before ES2017, we handled async work using:

- **Callbacks** → messy (callback hell)
- **Promises (.then)** → better, but still hard to read

async/await was introduced to **write asynchronous code that looks synchronous** — but it **does NOT block the thread**.

It is just **syntactic sugar over Promises**.

. What async Actually Does

When you write:

```
async function getData() {
  return "Hello";
}
```

JavaScript automatically converts it to:

```
function getData() {
  return Promise.resolve("Hello");
}
```

Every async function ALWAYS returns a Promise.

Even if you return:

- string
- number
- object

It becomes → Promise.resolve(value).

What await Actually Does

await pauses execution **inside the async function only**, not the whole JS thread.

```
const data = await fetchData();
```

Under the hood:

1. Calls fetchData() → gets a Promise
2. JS registers a continuation (.then)
3. Function exits to event loop
4. When Promise resolves → function resumes

Equivalent to:

```
fetchData().then(data => {
  // resume execution
});
```

So await is basically:

“Pause here, resume later when Promise resolves.”

```
console.log("Start");

async function run() {
  console.log("Inside");

  const data = await Promise.resolve("Done");
```

```
  console.log(data);
}

run();

console.log("End");
```

Output:

Start

Inside

End

Done

Inside an async function, when JavaScript hits await:

1. It **pauses that async function**.
2. It takes **all the code after the await** and wraps it internally like a .then(...) continuation.
3. That continuation is **NOT run immediately**.
4. JavaScript removes the async function from the Call Stack and continues executing the rest of the normal (synchronous) code.
5. When the awaited Promise resolves, that continuation is placed into the **Microtask Queue**.
6. The Event Loop waits until the Call Stack becomes empty.
7. Only then does it execute the microtask → which resumes the async function.

🔥 **Why "End" printed before "Done"?**

Because:

Step	What Happens
Start	Runs normally
Inside	Runs inside async
await	Function PAUSES
End	Event loop continues
Done	Microtask resumes async

👉 await **does not block JS** — it schedules continuation as a **microtask**.

Sequential vs Parallel Execution (Big Mistake Devs Make)

❌ **Sequential (Slow)**

```
const a = await fetchA();
const b = await fetchB();
const c = await fetchC();
```

This waits one-by-one.

Total Time = A + B + C

✅ **Parallel (Correct Way)**

```
const [a, b, c] = await Promise.all([
  fetchA(),
  fetchB(),
```

```
  fetchC()
]);
```

Now they run together.

Total Time = MAX(A, B, C)

Q. why await is used in async block only?

await needs a place where JavaScript can **pause execution and later resume it safely** — and only an async function creates that resumable environment.

1. What does async keyword do?

Answer:

- Makes a function always return a **Promise**.
- Wraps returned value in Promise.resolve(value).
- Allows use of await inside the function.

2. What does await do?

Answer:

- Pauses the async function execution.
- Waits for the Promise to settle.
- Schedules the remaining code as a **microtask**.
- Does **NOT block JavaScript thread**.